

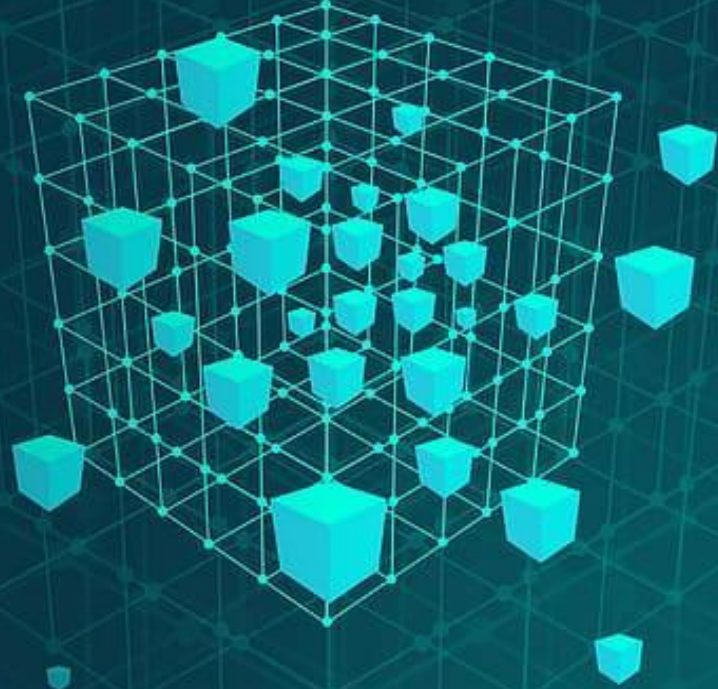
Project 3: Deep & Wide Nets, RNNs, Kernels & Regressions

**Neural Networks vs.
Classical Methods**

Tyler James
James Anderson
Shajeel Rehman

Data & Setup

- Two crash datasets (≥ 3000 rows each, ≥ 5 features)
- Collected one dataset via API
- Cleaning: missing values, outlier detection, categorical encoding, scaling.
- Feature engineering: interaction terms, ratios, rolling counts



Dataset A (API): NYC Motor Vehicle - Collisions (API).

Dataset B (GitHub): NYC Motor Vehicle - Vehicle records

```
Final Group 3 Project
File Edit View Insert Runtime Tools Help

Code (Dataset A via API: NYC collisions)

[2] ✓ 4s
# NYC Motor Vehicle Collisions from Socrata API (API dataset)
BASE = "https://data.cityofnewyork.us/resource/h9gl-nx95.json" # collisions API

# Choose fields we actually need (keep it compact but 25 features)
fields = [
    "crash_date", "crash_time", "borough", "zip_code", "latitude", "longitude",
    "number_of_persons_injured", "number_of_pedestrians_injured",
    "number_of_cyclist_injured", "number_of_motorist_injured",
    "contributing_factor_vehicle_1", "vehicle_type_code"
]

# Pull ~40k recent rows in batches of 5k (enough for 23k after cleaning)
rows_all = []
limit, offsets = 5000, list(range(0, 45000, 5000))
for off in offsets:
    params = {
        "$select": ".*".join(fields),
        "$order": "crash_date DESC, crash_time DESC",
        "$limit": limit,
        "$offset": off,
    }
    r = requests.get(BASE, params=params, timeout=120)
    if r.status_code != 200:
        print("HTTP", r.status_code, "at offset", off); break
    part = r.json()
    if not part: break
    rows_all.extend(part)
    print(f"Fetchd {len(part):5d} rows (offset {off}) | total {len(rows_all):5d}")
    if len(part) < limit: break

df_a_raw = pd.DataFrame(rows_all)
print("A raw shape:", df_a_raw.shape)
display(df_a_raw.head(0))

Fetchd 5000 rows (offset 0) | total 5000
Fetchd 5000 rows (offset 5000) | total 10000
Fetchd 5000 rows (offset 10000) | total 15000
Fetchd 5000 rows (offset 15000) | total 20000
Fetchd 5000 rows (offset 20000) | total 25000
Fetchd 5000 rows (offset 25000) | total 30000
Fetchd 5000 rows (offset 30000) | total 35000
```

```
Final Group 3 Project
File Edit View Insert Runtime Tools Help

Code (Dataset B from GitHub raw CSV)

[3] ✓ 1h
# Vehicles dataset from your GitHub (second dataset)
VEH_URL = "https://raw.githubusercontent.com/Btylerjames0/ADN660-03/refs/heads/main/vehicles_slim_2020plus.csv"
r = requests.get(VEH_URL, timeout=120); r.raise_for_status()
df_b_raw = pd.read_csv(pd.compat.StringIO(r.text), low_memory=False) if hasattr(pd.compat, "StringIO") else \
    pd.read_csv(__import__("io").StringIO(r.text), low_memory=False)
print("b raw shape:", df_b_raw.shape)
display(df_b_raw.head(0))
print("b columns:", list(df_b_raw.columns[:30]))

b raw shape: (50000, 24)
collision_id crash_date crash_time state_registration vehicle_type vehicle_make travel_direction vehicle_occupants driver_sex driver_license_status ... vehicle_damage_1 vehicle_damage_2 vehicle_damage_3 contributing_factor_1 cont
0 4267500 2020-01-01 2:44 NY Sedan HOND-CAR/SUV South 1.0 M Licensed ... Center Front End Right Front Bumper NaN Unspecified
1 4267000 2020-01-01 15:05 MI Sedan MITS-CAR/SUV East 1.0 F Licensed ... Right Rear Bumper No Damage No Damage Driver Inattention/Distractio
2 4267800 2020-01-01 15:05 NY Sedan BMW-CAR/SUV Unknown 1.0 F Licensed ... No Damage No Damage No Damage Unspecified
3 4363640 2020-01-01 6:20 NY Station Wagon/Sport Utility Vehicle MERZ-CAR/SUV North 0.0 NaN NaN ... Left Rear Quarter Panel Left Side Doors Left Front Quarter Panel Unspecified
4 4363648 2020-01-01 6:20 NJ Sedan NISS-CAR/SUV North 0.0 NaN NaN ... Left Side Doors Left Front Quarter Panel Left Front Bumper Unspecified
5 4363648 2020-01-01 6:20 FL Sedan INFI-CAR/SUV North 0.0 NaN NaN ... Right Front Quarter Panel Right Side Doors Right Rear Quarter Panel Unspecified
6 4381300 2020-01-01 23:07 NY Sedan INFI-CAR/SUV West 1.0 M Licensed ... Left Front Bumper Left Front Bumper Left Front Quarter Panel Driver Inattention/Distractio
7 4381300 2020-01-01 23:07 NY PK unk South 1.0 NaN NaN ... No Damage No Damage No Damage Driver Inattention/Distractio

8 rows x 24 columns
8 columns: ['collision_id', 'crash_date', 'crash_time', 'state_registration', 'vehicle_type', 'vehicle_make', 'travel_direction', 'vehicle_occupants', 'driver_sex', 'driver_license_status', 'driver_license_jurisdiction', 'pre_crash', 'point_of_i

Cleaning, EDA & Imputation (A & B)

Variables Terminal 9:49 AM Python 3
```

Linear & Regularized Models

	model	alpha	time_s	mse	label	l1_ratio
10	ElasticNet	0.001	7.691426	0.692149	Collisions	0.2
4	Ridge	10.000	0.562435	0.693154	Collisions	NaN
11	ElasticNet	0.001	6.136884	0.693525	Collisions	0.5
3	Ridge	1.000	0.625097	0.693908	Collisions	NaN
2	Ridge	0.100	0.466095	0.694874	Collisions	NaN
12	ElasticNet	0.001	12.117781	0.695111	Collisions	0.8
1	Ridge	0.010	0.491624	0.695131	Collisions	NaN
0	OLS	NaN	1.574717	0.695168	Collisions	NaN
5	LASSO	0.001	38.119981	0.695936	Collisions	NaN
13	ElasticNet	0.010	1.097546	0.702340	Collisions	0.2
	model	alpha	time_s	mse	label	l1_ratio
10	ElasticNet	0.001	11.948489	0.471405	Vehicles	0.2
4	Ridge	10.000	2.074966	0.471883	Vehicles	NaN
11	ElasticNet	0.001	4.184718	0.472528	Vehicles	0.5
12	ElasticNet	0.001	2.756463	0.473730	Vehicles	0.8
3	Ridge	1.000	2.118312	0.474515	Vehicles	NaN
5	LASSO	0.001	4.009344	0.474534	Vehicles	NaN
2	Ridge	0.100	2.077387	0.476113	Vehicles	NaN
1	Ridge	0.010	2.297608	0.476502	Vehicles	NaN
0	OLS	NaN	8.585690	0.476555	Vehicles	NaN
13	ElasticNet	0.010	3.927898	0.479319	Vehicles	0.2

- OLS, Ridge, LASSO, Elastic Net
- Hyperparameter tuning (alpha, lambda, etc.)
- Ridge improved stability; LASSO performed feature selection
- Baseline performance: R^2 lower than NN-based methods

Data background and context

- Crashes in NYC are much more common at night when visibility is low. Additionally, crashes between midnight and 6am tend to cause more severe or fatal injuries. This is why we used the engineered feature “hour”.

Citation: https://www.nyc.gov/html/dot/downloads/pdf/nyc_ped_safety_study_action_plan_technical_supplement.pdf

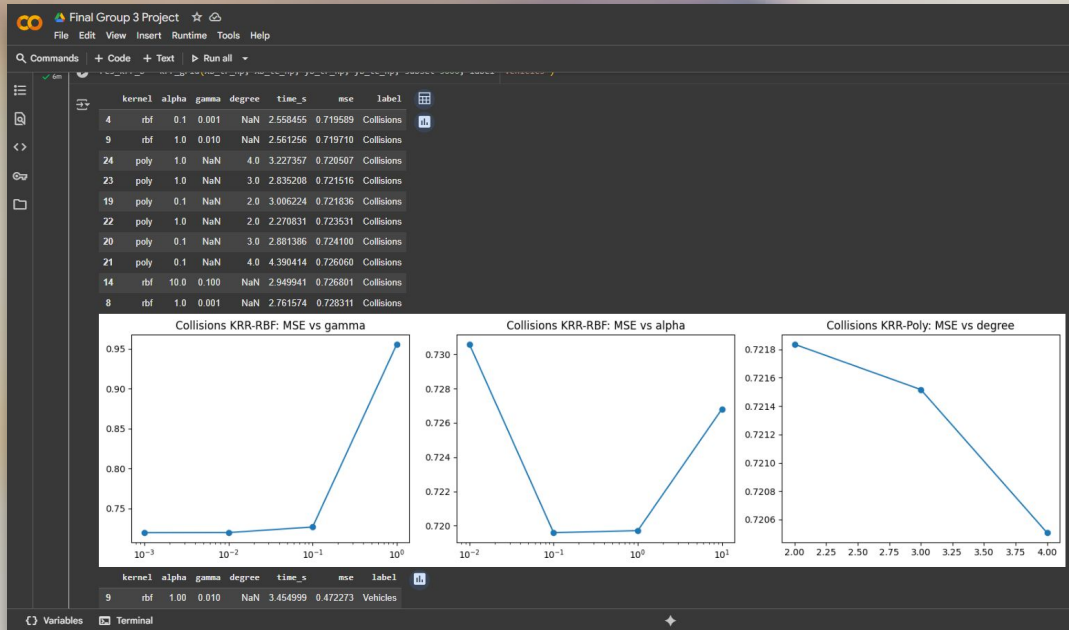
- Month of the year and seasonal cycles also directly contribute to the number of crashes. Earlier sunsets in Fall/winter correspond to more crashes, with accidents peaking in October and dropping to their lowest in march. This is why we used the engineered feature “month”

Citation: <https://injuryfacts.nsc.org/motor-vehicle/overview/crashes-by-month>

- Pedestrian crashes are more often than not at intersections: “Crashes at intersections typically comprise 51 percent of all fatalities and 68 percent of all injuries in a year. For pedestrians, the dangers are more pronounced: 59 percent of pedestrian fatalities and 77 percent of pedestrian traffic injuries occur at intersections.” This resulted in our engineered feature “is_intersection”

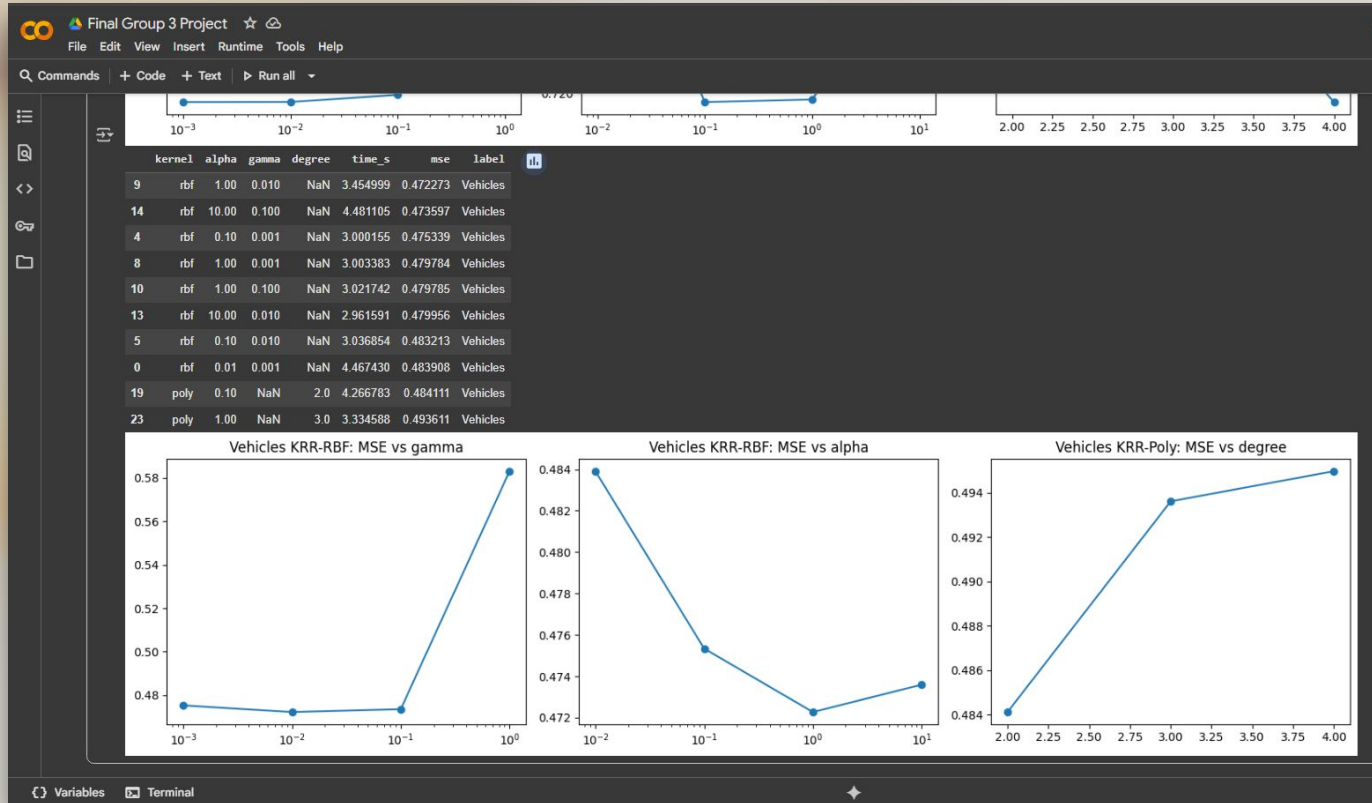
Citation: <https://www.nyc.gov/html/dot/html/pr2024/dot-enhances-pedestrian-safety.shtml>

Kernel Ridge Regression



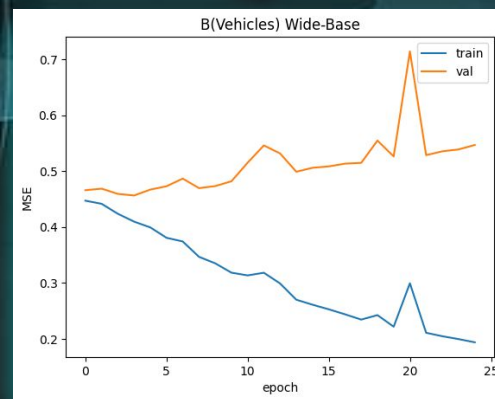
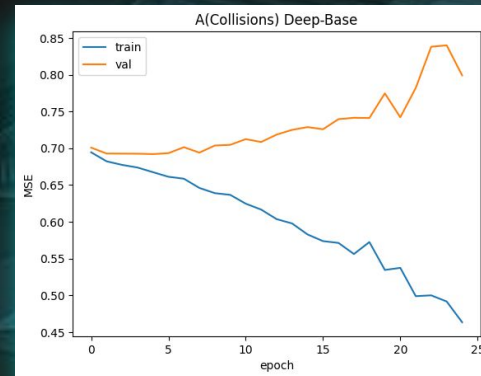
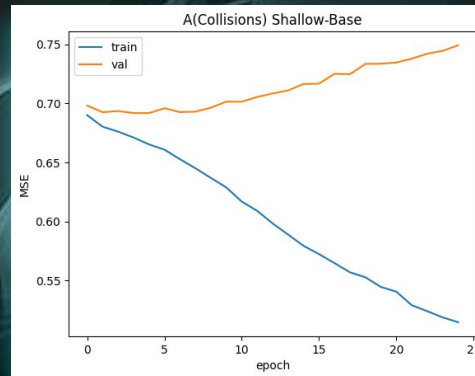
- Explored RBF & Polynomial kernels
- Manual grid search across gamma, degree, alpha
- Better capture of nonlinear relationships
- Higher compute cost but improved accuracy

Kernel Ridge Regression



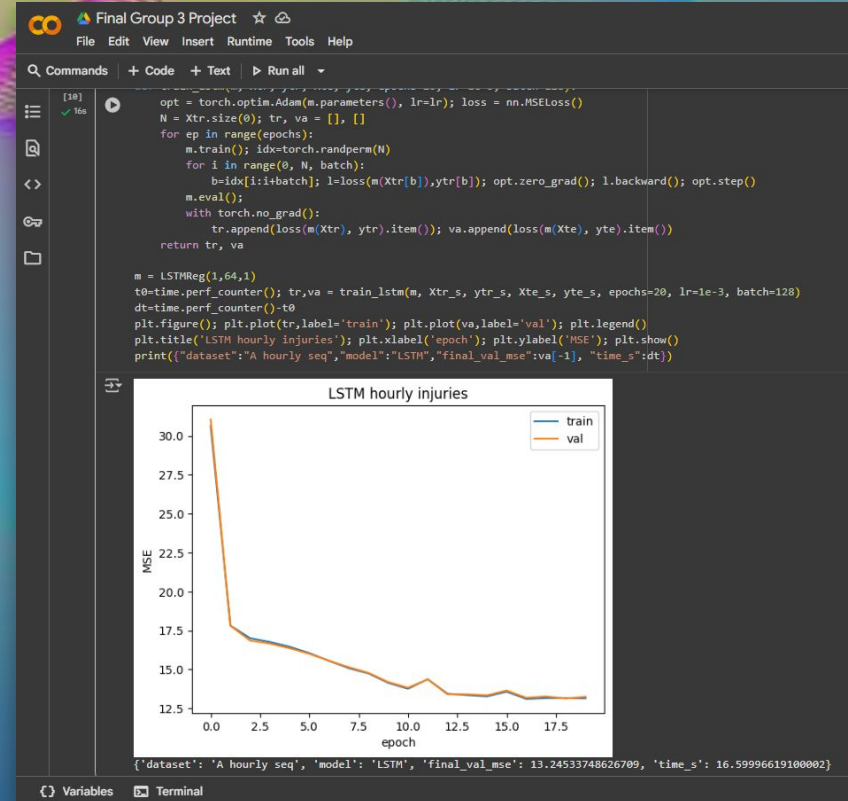
Neural Networks

- Shallow (1 hidden layer), Deep (4+ layers), Wide (single wide layer)
- Baseline vs. Enhanced (batch norm, dropout, custom activation)
- Improvements, but prone to overfitting vs. kernel methods



Recurrent Neural Networks

- Reframed crash data as sequences (time windows)
- Built LSTM regression for temporal prediction
- Converged but slower and less accurate than MLP/KRR
- Useful for capturing trends over time



Feature Transfer Experiment

- Extracted embeddings from NN penultimate layer
- Trained Ridge, RBF, Polynomial regressors on embeddings
- **Best model overall:** Kernel Ridge (RBF) on embeddings



The screenshot shows a Jupyter Notebook interface with a table of model performance metrics. The table is divided into two sections, one for 'label A' and one for 'label B'. Each section has a header row with columns: label, head, alpha, gamma, subset_n, time_s, and mse. The rows show different model configurations and their corresponding MSE values.

	label	head	alpha	gamma	subset_n	time_s	mse
3	A (Embed)	KRR_RBF_subset	1.0	0.01	5000	3.687186	0.715568
1	A (Embed)	KRR_RBF_subset	0.1	0.01	5000	2.291032	0.717506
0	A (Embed)	Linear	NaN	NaN	36000	0.027115	0.720329
4	A (Embed)	KRR_RBF_subset	1.0	0.10	5000	2.382046	0.720512
2	A (Embed)	KRR_RBF_subset	0.1	0.10	5000	2.268734	0.753575
	label	head	alpha	gamma	subset_n	time_s	mse
4	B (Embed)	KRR_RBF_subset	1.0	0.10	5000	2.346477	0.487425
3	B (Embed)	KRR_RBF_subset	1.0	0.01	5000	3.556467	0.487503
1	B (Embed)	KRR_RBF_subset	0.1	0.01	5000	2.401859	0.488382
0	B (Embed)	Linear	NaN	NaN	40000	0.045469	0.492797
2	B (Embed)	KRR_RBF_subset	0.1	0.10	5000	2.427613	0.505858

Final Results and Reflections

- **Hardest issue:** The hardest issue was the fact that the website the API was being pulled from was undergoing maintenance while trying to rerun the project.
- **New insight:** Combining neural embeddings with classical regressors can outperform standalone deep nets.
- Final deliverables uploaded:
 - RESULTS_SUMMARY.md
 - model_results_summary_all.csv
 - model_comparison_all.png



THANK
YOU!